Technical Evaluation Report

JAVA, REACT & ORACLE DATABASE Assessment

Candidate:	giddaluru.prakash@irissoftware.com
Email:	giddaluru.prakash@irissoftware.com
Tech Stack:	JAVA, REACT & ORACLE DATABASE
Completed:	September 03, 2026 at 08:02 PM
Report Generated:	September 04, 2026 at 12:51 PM

Overall Assessment

75.9 / 100

Evaluation Duration

Metric	Value
Actual Time Taken	104.5 minutes
Planned Duration	125 minutes
Time Efficiency	83.6% of planned time ✓

Category Scores

Category	Score
SQL Fundamentals	61.0/100
Core Java	44.0/100
JUnit	63.0/100
Database	84.0/100
Fundamentals & Architecture	100.0/100

React:HTML5	82.0/100
PL-SQL	68.0/100
Spring	52.0/100
Data Structures & Algorithms	38.0/100
React:Fundamentals	82.0/100
React:JS-ES6+	68.0/100
React:Hooks	100.0/100
Spring Boot	74.0/100
Programming(java)	74.0/100
React:Redux	78.0/100
Concurrency & Multithreading	82.0/100
SQL & Data Handling	100.0/100
Microservices	74.0/100
Performance & Security	82.0/100
React:Optimization	62.0/100
Advance SQL	88.0/100
React:Maps	82.0/100
Memory Management	74.0/100
React:TanStack Query	82.0/100
JVM	88.0/100
API Design	78.0/100
Design Patterns	88.0/100

Communication Assessment

Metric	Score

Confidence Level	6.5/10
English Proficiency	6.4/10
Clarity Score	6.0/10

Strengths

• Uses the correct recursive CTE pattern

• Correctly identified clustering as a primary optimization

• Disconnects the previous observer before creating a new one

• Uses functional state update for page increment

• Correctly connected associative arrays with key-based rule lookup

Areas for Improvement

• State more explicitly that the pattern lets one component own logic/state while letting the consumer control the rendered output

• Make the comparison more explicit by stating that CountDownLatch is for waiting until a count reaches zero, while CyclicBarrier is for a fixed group of threads all waiting at the same synchronization point.

• Structure the answer more clearly: first define optimistic updates, then explain implementation step by step

• Ensure the solution works for all valid inputs, especially n=1

• Use the standard terminology such as 'mutants', 'killed mutants', and 'surviving mutants' to demonstrate clearer understanding.

Detailed Question Feedback

Q1: Please explain your most recent project in detail. Describe the technologies used, your role and responsibilities, the challenges you faced, and how you overcame them. Also explain the architecture and any significant technical decisions you made.

Category: Project Experience | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

> Okay, coming to Arctech events driving involvement microservices using Java 21 Spring Boot, AWS, SQS, RADIS and PostgreSQL SQS process like around 150k plus daily transactions while maintaining 99.99% availability API literacy and architecture retrieve and their failures handling workflow and the development a react-based dashboard for the monitoring and manual reprocessing reducing manual intervity by 30% and improving turnaround time by 25%

Good: Correctly identified technologies used, Mentioned high availability (99.99%)

Improve: Provide a clear explanation of the architecture and significant technical decisions made, Discuss challenges faced and how they were overcome in more detail, Elaborate on API literacy and architecture points

Detailed Feedback:

> The candidate's answer provides a good overview of their recent project but lacks some key technical details and clarity. They correctly identified the technologies used (Java 21, Spring Boot, AWS, SQS, Redis, PostgreSQL) and their

role in maintaining high availability (99.99%). However, they failed to provide a clear explanation of the architecture and significant technical decisions made. The candidate also mentioned API literacy and architecture but did not elaborate on these points. Additionally, they missed discussing challenges faced and how they overcame them.

Q2: What is a view?

Category: SQL Fundamentals | Question Category: Foundational | Difficulty: easy | Type: VERBAL

Candidate's Verbal Answer:

> Okay, so coming to view means a view is we are using for the SQL is a virtual table based on the results set for a queue queue and in coming to the Java backend it will be it will be using main thing for the most important for you yours as a virtual table made from SQL query it does not stored data it shows data from real table and we are used to simply and secure data for access and coming to it will it is presently layer that display modes data and coming to for example we are taking for the SQL code means we are using for the use MVC view in restful for the spring boot in MVC pattern mode equal to like data views equal to what user seems controller equal to logic yeah for these all we are using for the view

Score: 61/100

Good: Captured the central definition of a database view as a virtual table, Correctly noted that the data is shown from real/base tables rather than stored in the view, Mentioned practical purposes such as simplification and security

Improve: Keep the answer strictly in the Oracle Database context; the discussion of Java backend, REST, Spring Boot, and MVC views is unrelated here., State the definition more precisely: in Oracle, a view is a database object created from a SELECT statement that presents data from one or more tables or views., Avoid vague phrases like 'presentation layer' unless clearly tied back to the database concept, because that makes the answer sound like UI architecture rather than Oracle SQL.

Detailed Feedback:

The question only asked for a definition/explanation of what a view is in Oracle Database. The candidate did provide the core idea: a view is a virtual table based on the result set of a SQL query, it does not store data itself, and it can be used to simplify and secure access to underlying table data. Those are the main required points, and they are technically correct for Oracle views in general. However, the answer becomes muddled by mixing in unrelated Java/Spring MVC 'view' concepts, which is not relevant to Oracle Database and weakens the precision of the response. The candidate also did not clearly state that a view is a stored SQL query/object in the database that presents data from one or more tables. So overall, this is a basically correct but somewhat confused answer: enough to show understanding of the database concept, but not cleanly scoped to Oracle.

Q3: What is method reference and its types?

Category: Core Java | Question Category: Core | Difficulty: easy | Type: VERBAL

Candidate's Verbal Answer:

> okay what is method reference and its types okay coming to what is method of reference means it's it is a shortcut for lambda and its types means where we have types of method reference one is static method reference and another one is constructed reference yeah that's it

Score: 44/100

Good: Recognized that method references are connected to lambda expressions, Correctly identified static method reference as one type, Attempted to identify constructor reference as a type

Improve: Define method reference more accurately: it is a way to refer to an existing method or constructor using the :: operator, often used when a lambda only calls that method., List all standard Java method reference types, not just two: static method reference, instance method reference of a particular object, instance method reference of an arbitrary object of a particular type, and constructor reference., Use the correct term 'constructor reference' instead of 'constructed reference'.

Detailed Feedback:

The candidate shows partial understanding but the answer is incomplete for the question asked. In Java, a method reference is indeed a shorthand/compact form related to lambda expressions, used to refer to an existing method or constructor when the lambda body simply calls that method. So the core idea of 'shortcut for lambda' is partially correct, though imprecise. However, the candidate's answer on types is significantly incomplete. They mentioned 'static method

reference' and 'constructor reference' only. The standard Java method reference types are: static method reference (ClassName::staticMethod), particular object instance method reference (instanceRef::instanceMethod), arbitrary object of a particular type instance method reference (ClassName::instanceMethod), and constructor reference (ClassName::new). Because the question explicitly asked for the types, missing half of them is a major gap. There are no major fabricated terms, but 'constructed reference' is not the standard term; the correct term is 'constructor reference'. Overall, this is a below-average answer: some correct core understanding, but incomplete and not technically complete for the requested scope.

Q4: What is mutation testing and how does it improve test quality?

Category: JUnit | Question Category: Foundational | Difficulty: hard | Type: VERBAL

Candidate's Verbal Answer:

> Mituation testing is a technical to evaluate test quality by injections, small falls into code and checking if test catch, if it improves test quality behind line coverages by ensuring test actually detect bugs. For example, I am telling for that it will be a benefits in our project that is like finding missing test cases, preventive bugs in the production and good for Kevin storages and it will be coming improve for the interview goal. Yeah, this all we are using for that.

Score: 63/100

Good: Answered the actual question rather than going off-topic, Captured the main purpose of mutation testing correctly, Correctly connected mutation testing to improving test quality beyond coverage metrics, Mentioned practical benefit of identifying missing tests

Improve: Define the mechanism more precisely: mutation testing creates small code changes (mutants) and checks whether the test suite fails against them., Explain how it improves test quality more clearly: surviving mutants show that tests may execute code but not assert behavior strongly enough., Use the standard terminology such as 'mutants', 'killed mutants', and 'surviving mutants' to demonstrate clearer understanding.

Detailed Feedback:

The candidate shows a basic but real understanding of mutation testing. Interpreting the transcription, they are saying that mutation testing evaluates test quality by injecting small faults into code and checking whether the tests catch them, and that this goes beyond line coverage by verifying tests can actually detect bugs. That is substantially correct and directly answers both parts of the question at a basic level. They also mention benefits such as finding missing test cases and preventing bugs from reaching production, which aligns with how mutation testing improves test quality. However, the answer is incomplete and somewhat imprecise. A stronger answer should clearly state that mutation testing creates small code changes called mutants, runs the test suite, and measures whether tests fail (kill the mutant) or still pass (surviving mutant), with surviving mutants indicating weak or missing assertions/test cases. The candidate did not clearly explain the mutant/kill-survive mechanism, which is a key part of how mutation testing works. Some phrases near the end are unclear or likely mistranscribed (for example, "Kevin storages" and "interview goal"), so they do not add meaningful technical value. Overall, this is an average answer: technically relevant, mostly correct in substance, but lacking precision and completeness on the core mechanism.

Q5: Write a recursive CTE to display employee hierarchy.

Category: Database | Question Category: Foundational | Difficulty: medium | Type: CODE

Candidate's Code Answer:

```
WITH RECURSIVE EmployeeHierarchy AS(--Anchor Member : Select top-level managers(where manager_id
is NULL)
SELECT emp_id,emp_name,manager_id, 1 AS level FROM employees WHERE manager_id IS NULL
UNION ALL
--Recursive Member: Join employees with the CTE to find direct reports
SELECT e.emp_id,e.emp_name,e.manager_id,
h.level + 1 AS level
FROM employees e INNER JOIN EmployeeHierarchy h ON e.manager_id = h.emp_id)
--Final Selection
SELECT emp_id,emp_name,manager_id,level
FROM EmployeeHierarchy ORDER BY level,emp_id;
```

Score: 84/100

Good: Uses the correct recursive CTE pattern, Generic solution with no hardcoded values, Correctly increments hierarchy depth with level + 1, Explicit column selection and reasonable final ordering, Handles multiple top-level managers naturally

Improve: Add cycle detection to prevent infinite recursion on bad data, Consider handling orphaned employees if they should appear in the hierarchy, Build a path column if hierarchical display order matters beyond simple level sorting, Be aware of recursion depth limits in the target SQL engine

Detailed Feedback:

This is a solid generic recursive CTE solution for displaying an employee hierarchy. It correctly uses an anchor member to start from top-level employees where manager_id IS NULL, then recursively joins employees to prior hierarchy rows via manager_id = emp_id and increments the level. That is the standard approach for this problem and it is not hardcoded to any example data. The main limitation is completeness around edge cases. If the data contains cycles in the management chain, this query can recurse indefinitely or until the database recursion limit is hit. It also assumes that top-level hierarchy roots are exactly those with manager_id IS NULL; if the schema allows orphaned employees whose manager_id references a missing employee, they will be excluded entirely. Depending on the SQL dialect, recursion depth limits may also affect deep hierarchies. Ordering by level, emp_id is acceptable for display, but it does not preserve a true tree traversal order. If the requirement is only to display hierarchy levels, this is fine. If the expectation is a manager-subordinate path ordering, the query would need a path column. Overall: correct core logic, clean structure, and good SQL usage, but not fully robust for problematic data.

Q6: An Oracle Instance consists of:

Category: Fundamentals & Architecture | Question Category: Foundational | Difficulty: easy | Type: OBJECTIVE

Candidate's Verbal Answer:

Memory structures and background processes

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'Memory structures and background processes' is correct.

Q7: What is the difference between inline, block, and inline-block elements?

Category: React:HTML5 | Question Category: Foundational | Difficulty: easy | Type: VERBAL

Candidate's Verbal Answer:

Ok, first coming to inline, inline takes only content width stays in same line, does not allow width height, example span and coming to block, it will be takes full width starts on new line, allow width height, example div, dev, p and coming to inline block, stay in small line like inline but allows width height like block.

Score: 82/100

Good: Answered the exact comparison the question asked, Technically mostly correct across all three display types, Used valid HTML element examples, Showed clear basic understanding of layout behavior

Improve: State the differences more precisely: block elements start on a new line and by default expand to available width, while inline elements flow within text and width/height typically do not apply in the normal way., Clarify inline-block as: it stays in the same line like inline, but behaves like a block-level box for sizing.

Detailed Feedback:

The candidate gave a solid and mostly correct comparison of inline, block, and inline-block elements, which is exactly what the question asked. They correctly stated that inline elements take only the content width and stay on the same line, and that block elements take full available width, start on a new line, and allow width/height. They also correctly described inline-block as behaving inline in flow while still allowing width and height like a block. The examples given (span for inline, div and p for block) are appropriate. There are minor phrasing issues from speech transcription, but the technical meaning is clear. The main limitation is that the explanation is brief and slightly imprecise in wording—for example, inline elements do not generally accept width/height in the same way block/inline-block do, but there are nuances depending on replaced elements. However, for the scope of this question, the answer is accurate and sufficiently complete. This is a good answer, not exceptional, because it covers the core differences correctly but

without much precision or polish.

Q8: Explain how to implement table-driven logic using associative arrays.

Category: PL-SQL | Question Category: Core | Difficulty: hard | Type: VERBAL

Candidate's Verbal Answer:

Normally we write if status equal to p then pending else if which is slow and hard to maintain in the table drive approach I define like t I mean like type t tap is table of wire car to index by wire car to unload all business rules from a configuration table into that associative arrays only then in the main logic I just do arrays key to get the values no condition if business rules changes I just update the configure I mean like config table no code changes needed it's heavily used for discounts status mapping validation it much faster especially in bulk process if I have to say the coding means like if I have to tell the example scenario so coming to type t rule is table of wire car in array 100 IDE we are using for the rules pending for the using status yeah

Score: 68/100

Good: Understood the main purpose of table-driven logic, Correctly connected associative arrays with key-based rule lookup, Recognized the role of a configuration table in externalizing business rules, Gave practical business-rule examples rather than speaking only abstractly, Demonstrated some Oracle-specific PL/SQL knowledge despite transcription issues

Improve: Explain the implementation more clearly as a sequence: define associative array type, declare variable, load rows from config table, then perform keyed lookups in the logic., State the Oracle syntax more accurately, e.g. `TYPE t_rule_tab IS TABLE OF VARCHAR2(100) INDEX BY VARCHAR2(30);` or similar, instead of fragmented type definitions., Mention safe access patterns such as checking `array.EXISTS(key)` before reading a value., Clarify that the table stores the configurable rules permanently, while the associative array is an in-memory PL/SQL structure used at runtime.

Detailed Feedback:

The candidate shows a real, practical understanding of table-driven logic in Oracle using associative arrays, despite the heavy transcription noise. The core meaning is clear: instead of hard-coded IF/ELSE branches, store rules in a configuration table, load them into a PL/SQL associative array, and perform lookups by key in the main logic. That is a valid explanation of how table-driven logic can be implemented in Oracle. Technically, they correctly conveyed several important points: using a PL/SQL type such as `TYPE t_rule IS TABLE OF VARCHAR2(...) INDEX BY VARCHAR2(...)`, loading rule data from a table into the associative array, and then using key-based access in procedural logic. They also correctly connected this to maintainability and avoiding code changes when business rules change. However, the answer is only moderately complete for the question 'Explain how to implement table-driven logic using associative arrays.' It explains the idea and gives fragments of implementation, but it is not well structured and misses some important implementation details that would make the explanation complete in Oracle terms. For example, they did not clearly explain the full pattern: define the associative array type, declare a variable, populate it from a configuration table (typically via a cursor/loop or BULK COLLECT into intermediate structures), and then use `EXISTS` / keyed lookup safely in the logic. They also did not clearly distinguish that associative arrays are PL/SQL-only structures, while the actual rules persist in database tables. The statement that this is 'much faster especially in bulk process' is plausible in the sense that in-memory lookups can reduce repeated branching or repeated SQL access, but it was asserted loosely without clarifying that performance depends on how the data is loaded and used. There are also transcription-related terminology issues such as 'wire car' which is clearly intended to mean `VARCHAR2`, and 'index by wire car' meaning `INDEX BY VARCHAR2(...)`. I did not penalize those as language errors because the intended Oracle concept is recognizable. Overall: technically meaningful and mostly correct, but incomplete and poorly articulated. This is an average answer leaning toward good, not a strong one.

Q9: Explain query hints and how to use them for performance tuning.

Category: Spring | Question Category: Core | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

We are using for the performance tuning it will be main thing for the optimized to override its default exception planning. Optimizer using choose the best plan based on statistic but when the statistics are stale or data in seek it can choose a wrong plan then we use a hint to force a better plan for the performing tune and also I have to tell for the optional I mean for the types of modes means it will be showing for the access path hint and join hints and parallelize these all we are using for this.

Score: 52/100

Good: Demonstrated real understanding that hints are related to execution plan control, Correctly connected poor plans to stale statistics and skewed data, Mentioned valid hint categories relevant to performance tuning

Improve: Define query hints more clearly: they are directives given to the ORM or database engine to influence query execution behavior for performance-related reasons., Answer the 'how to use them' part explicitly in Java terms, such as applying JPA/Hibernate hints through Query.setHint(...) or @QueryHint, and noting that some hints are provider-specific., Clarify that hints do not always 'force a better plan'; they influence or direct execution, and should be validated with profiling/explain plans because they can become counterproductive over time., Use more precise terminology: 'execution plan' instead of the transcribed 'exception planning'.

Detailed Feedback:

The candidate shows partial understanding of query hints at a database/SQL level: they correctly convey that hints can be used to override the optimizer's default execution plan choice when statistics are stale or data distribution/skew causes a poor plan, and they mention categories such as access path hints, join hints, and parallelism hints. That is meaningful technical content and does address the core idea of what query hints are and why they are used for performance tuning. However, the answer is incomplete and somewhat imprecise for the question asked. In a Java-context interview, query hints are commonly discussed in relation to JPA/Hibernate as well as underlying SQL/database optimizer hints. The candidate stayed at a generic database optimizer level and did not explain how they are actually used from Java code (for example via JPA Query/TypedQuery setHint, @QueryHint, or provider-specific hints). Since the question explicitly asked both to explain query hints and how to use them for performance tuning, the lack of any concrete usage mechanism is a significant gap. There are also wording issues that obscure meaning: 'default exception planning' is almost certainly a transcription error for 'execution planning', and 'data in seek' likely means data skew. Interpreting generously, the intended meaning is understandable. Still, the answer remains broad and underdeveloped. A stronger answer would have clearly defined query hints as directives supplied to the ORM or database optimizer to influence execution behavior, noted that they should be used sparingly after measuring performance, and explained practical usage in Java/JPA such as fetch size, timeout, read-only/cache hints, fetch graph/load graph, or provider-specific SQL hints depending on the stack. Overall: partial technical correctness, but incomplete relative to the full scope of 'how to use them' in a Java setting.

Q10: Given a positive integer n, generate an n x n matrix filled with elements from 1 to n^2^ in spiral order. Example 1: Question image Input: n = 3 Output: [[1,2,3],[8,9,4],[7,6,5]] Example 2: Input: n = 1 Output: [[1]] Constraints: 1 <= n <= 20

Category: Data Structures & Algorithms | Question Category: Foundational | Difficulty: medium | Type: CODE

Candidate's Code Answer:

```
public class Main{
public static void main(String[]args){
int n=3;
int[][] m = new int[n][n];
int t=0,b=n-1,l=0,r=n-1,k=1;
while(k<=n*n){
for(int i=l;i<=r;i++) m[t][i]=k++; t++;
for(int i=t;i<=b;i++) m[i][r]=k++; r--;
for(int i=r;i>=l;i--) m[b][i]=k++; b--;
for(int i=b;i>=t;i--) m[i][l]=k++; l++;
}
}
}
```

Score: 38/100

Good: Uses the standard boundary-based spiral traversal approach, Fills values incrementally from 1 upward, Time and space complexity are appropriate for the problem

Improve: Replace the hardcoded n=3 with a parameterized solution, Return the generated matrix instead of only filling a local variable in main, Add boundary checks before executing the bottom-row and left-column traversals, Ensure the solution works for all valid inputs, especially n=1, Structure the code as a reusable method expected by coding platforms

Detailed Feedback:

This is implementation code, not a test. The core spiral-fill idea is recognizable and mostly correct for the hardcoded case n=3, but the submission does not actually solve the stated problem as a reusable solution. The code hardcodes n=3 inside main, never accepts input, never returns the matrix, and never outputs anything. That makes it incomplete relative to the problem requirement. Algorithmically, the boundary approach (top/bottom/left/right with incrementing k) is standard. However, the loop body is unsafe because each directional pass runs unconditionally. In spiral generation, after shrinking boundaries, you must guard the bottom-row and left-column traversals with boundary checks such as top <= bottom and left <= right. Without those checks, the code can overwrite cells in some iterations. For example, with n=1, after filling the top row, t becomes 1, r becomes -1 later, and the third pass writes m[0][0] again with 2 before the while loop exits. So even within the stated constraints, it fails on a valid input. There are also Java-specific quality issues: the class name Main is acceptable for a runnable snippet, but this is not structured as a problem solution method; formatting is poor; and the code is not usable by an online judge expecting a method like generateMatrix(int n). Correct approach: implement a method returning int[][], parameterize by n, and add boundary guards before the bottom-row and left-column traversals.

Q11: What are render props and when would you use them?

Category: React:Fundamentals | Question Category: Core | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

RenderProp is a pattern where components take a function as a prop that returns jsx and it calls that function to decide what to render. It is used for sharing logic, stateful behavior between components and coming to use it when you need to reuse same stateful logic in multiple components like mouse, tracking, data fetching, toggle logic without duplication code.

Score: 82/100

Good: Technically accurate core definition, Correctly identified the main purpose: sharing reusable stateful logic, Answered both parts of the question: what it is and when to use it, Provided valid React-relevant examples

Improve: Clarify that a render prop is any function prop used by a component to render content, not only a prop literally named `render`, State more explicitly that the pattern lets one component own logic/state while letting the consumer control the rendered output

Detailed Feedback:

The candidate gave a solid and technically accurate answer to what was asked. They correctly defined render props as a pattern where a component receives a function prop that returns JSX, and the component invokes that function to determine what to render. They also correctly explained when to use it: to share reusable stateful logic/behavior across multiple components without duplicating code. The examples they mentioned—mouse tracking, data fetching, and toggle logic—are valid classic use cases in React. What keeps this from scoring higher is that the definition was slightly narrow and informal. A stronger answer would explicitly note that the function prop is often called during render and that the pattern is specifically about delegating rendering to the consumer while encapsulating reusable logic in the provider component. Also, they did not mention that the prop does not have to literally be named `render`; any prop whose value is a function used to render UI fits the pattern. Still, those are refinements rather than major omissions. Overall, this is a good answer: accurate, relevant, and complete enough for a definition-plus-usage question.

Q12: Write a function to flatten a deeply nested array.

Category: React:JS-ES6+ | Question Category: Core | Difficulty: hard | Type: CODE

Candidate's Code Answer:

```
import {useState} from "react";
function App(){
const [arr] = useState([1,[2,[3,[4]]]]);
const flatten =(arr)=>
arr.reduce((acc,val) =>
acc.concat(Array.isArray(val) ? flatten(val) : val),[]);
return <div>{flatten(arr).join(", ")} {/*1,2,3,4*/}</div>
}
```

Score: 68/100

Good: Uses a generic recursive approach rather than hardcoded example-specific logic, Correctly handles arbitrary nesting depth for normal array inputs, Concise implementation with standard JavaScript array methods, Produces the

expected flattened output for the provided sample

Improve: Provide a standalone flatten function instead of embedding it in a React component, Remove unnecessary useState and React-specific code, Handle invalid or unexpected inputs more safely, Consider avoiding repeated concat allocations if performance matters on large inputs

Detailed Feedback:

The core flattening logic is correct for nested arrays of arbitrary depth. The recursive reduce + concat approach will transform [1,[2,[3,[4]]]] into [1,2,3,4], and it generalizes beyond the example rather than hardcoding the sample output. However, this submission does not really present a standalone function as requested; it wraps the logic inside a React component and uses useState unnecessarily. That makes the solution less complete and less reusable for the actual problem statement. There is also no handling for invalid inputs such as null, undefined, or non-array values, so it assumes a valid array is always passed. From a React perspective, importing state just to hold a constant array is poor practice and adds noise. Algorithmically, it works, but as a solution to the prompt it is more of a demo inside a component than a clean implementation of a flatten function. ■■ Note: Compilation failed due to environment/infrastructure issues (e.g. missing JUnit/Spring jars in the runner). No score penalty applied. Compiler output: Code execution not supported for language: javascript. Evaluation will be done by LLM only.

Q13: What is the initial value parameter in useState?

Category: React:Hooks | Question Category: Core | Difficulty: easy | Type: OBJECTIVE

Candidate's Verbal Answer:

Optional

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'Optional' is correct.

Q14: How does Spring AOP work? Explain pointcuts, advice, and aspects.

Category: Spring Boot | Question Category: Core | Difficulty: hard | Type: VERBAL

Candidate's Verbal Answer:

Spring AOP works via runtime proxy. It creates a proxy object around your bin. When you call a method proxy, interacts it, runs extra logic, then calls original method. Coming to explain means, expects, classic with a cross cutting logic. Example, Earth rate, expects, plus Earth rate compounds. Coming to advise, it's what to do and when to do. Type are before running, before method. After running, after method, around before and after most powerful, after written, after throwing. Point cuts it where to apply that advice and expression to match methods. Example, applying logic, I mean, plugins to all methods of services packages. Coming to joint point, it's the specific point where advice is applied like a method execution.

Score: 74/100

Good: Correctly explains the core proxy-based mechanism of Spring AOP, Covers all three requested terms: pointcuts, advice, and aspects, Correctly identifies advice timing categories, Shows practical understanding that pointcuts target matched methods

Improve: Define aspect more clearly: an aspect is a modularization of a cross-cutting concern, typically combining pointcuts and advice., Clarify that in Spring AOP, the join points supported are generally method executions on Spring-managed beans., Be more precise when listing advice types: before, after, around, after returning, and after throwing.

Detailed Feedback:

The candidate demonstrates solid basic understanding of Spring AOP and answers the core of what was asked: how Spring AOP works, and the meanings of advice, pointcuts, and aspects. They correctly explain that Spring AOP is proxy-based at runtime, with a proxy wrapping a bean and intercepting method calls to run extra logic before delegating to the target method. They also correctly describe advice as the action taken at certain times, and pointcuts as expressions that select where advice applies. The answer also mentions join points, which is relevant supporting knowledge. However, the answer is somewhat incomplete and imprecise in a few places. The explanation of aspect is

garbled by transcription, but the intended meaning appears to be that an aspect encapsulates cross-cutting logic, which is correct. The candidate lists advice types and appears to mean before, after, around, after returning, and after throwing; that is broadly correct, though the phrasing is rough. A key limitation is that they do not clearly state that in Spring AOP, join points are typically method executions only, unlike full AspectJ. Also, while they explain the proxy mechanism, they do not mention that Spring uses JDK dynamic proxies or CGLIB proxies depending on the target, which would have made the explanation stronger, though this is not strictly required for a complete answer. Overall, this is a good answer for the scope of the question: technically mostly correct, covers the requested concepts, but lacks precision and polish in defining aspect and in clarifying Spring-specific limitations.

Q15: What is the difference between dependencyManagement and dependencies in Maven?

Category: Programming(java) | Question Category: Foundational | Difficulty: easy | Type: VERBAL

Candidate's Verbal Answer:

> Dependency is nothing but actually added the dependencies to our project. It will be downloaded and including in classic path. Classic path, if you have put it in parent form, all children get it. Coming to dependency management does not add dependencies, it just version controller central management, it's only declared if any child using the dependencies use the version. Child still has to declare the dependencies in its own dependencies but without versions. Coming to dependency mavin, dependency mavin.

Score: 74/100

Good: Captured the main conceptual difference correctly, Understood that `dependencyManagement` is declarative and does not automatically bring in the dependency, Correctly identified the child-module behavior of declaring dependencies without versions

Improve: State more precisely that `dependencies` means the module actually uses that artifact, while `dependencyManagement` only defines default dependency information to be used when that dependency is declared., Clarify the parent/child point: `dependencyManagement` centralizes versions for children, whereas `dependencies` in a module means that module inherits/uses those dependencies through Maven inheritance, which is a different mechanism., Avoid incomplete trailing phrases and repetition so the comparison is more direct and structured.

Detailed Feedback:

The candidate demonstrates a solid understanding of the core difference between Maven `dependencies` and `dependencyManagement`. They correctly explain that entries under `dependencies` are actually added to the project and end up on the classpath, while `dependencyManagement` does not itself add the dependency but centrally manages the version so child modules can declare the dependency without repeating the version. They also correctly mention the common parent/child usage pattern in multi-module builds. However, the answer is somewhat fragmented and repetitive, and a few statements are imprecise. For example, saying that if a dependency is put in the parent `dependencies`, all children get it is not generally correct in the same way as `dependencyManagement`; dependencies are inherited in Maven POM inheritance, but the distinction should have been stated more clearly, especially that `dependencyManagement` provides defaults/constraints and `dependencies` actually includes the artifact for that module. The ending trails off and does not add value. Overall, this is a good answer for a definition/comparison question: it covers the main conceptual difference accurately enough, but lacks precision and polish.

Q16: What is the use of Redux?

Category: React:Redux | Question Category: Core | Difficulty: easy | Type: VERBAL

Candidate's Verbal Answer:

> Redex is a centralized state management library. Use it when you need same data in many components or state in complex and shared across the apps. When we use it, coming to a single source of truth, pull app state in one storage and coming to second avoids pro-drillings. No need to pass data from parent to child to child.

Score: 78/100

Good: Answered the actual question directly, Correctly identified Redux as useful for shared and complex state, Correctly connected Redux usage with avoiding prop drilling, Demonstrated practical understanding of why Redux is used in React apps

Improve: State more precisely that Redux is used for centralized and predictable application state management through a store, Make the explanation more structured and less fragmented so the core purpose is clearer

Detailed Feedback:

The candidate gave a solid and mostly correct answer to what was asked. The question only asks for the use of Redux, and they correctly explained that Redux is used for centralized state management in React applications, especially when the same data is needed across many components or when state is complex and shared across the app. They also correctly referenced the idea of a single source of truth and reducing prop drilling by avoiding passing data through multiple parent-child levels. There are some language/transcription issues ("Redex," "pull app state in one storage," "coming to second"), but the intended technical meaning is clear. Technically, the answer is accurate enough for the scope of the question. What keeps this from a higher score is that the explanation is somewhat fragmented and imprecise. Redux is not just "one storage" in casual terms; more precisely, it provides a centralized store for predictable state management. Also, while avoiding prop drilling is a common reason to use Redux, that alone is not the full purpose of Redux. Still, for a definition/explanation question, the candidate covered the main use correctly.

Q17: How do CountDownLatch and CyclicBarrier differ?

Category: Concurrency & Multithreading | Question Category: Foundational | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

coming to countdown latch one time use one count reach zero you can't reuse it one more one or more threads wait for others threads to finish example main thread waits for three services thread to complete and coming to cyclic bearer it's reusable once all threads reach bearer it resets and can be used again that's why it's called cyclic all threads wait for each other to reach a point means it's a common point and coming to example three threads processing all must reach checkpoint before moving forward

Score: 82/100

Good: Captured the primary semantic difference accurately, Used correct practical examples to illustrate each construct, Showed real understanding of synchronization behavior rather than just naming APIs, Answer stayed focused on the comparison asked

Improve: Make the comparison more explicit by stating that CountDownLatch is for waiting until a count reaches zero, while CyclicBarrier is for a fixed group of threads all waiting at the same synchronization point., Mention that in CyclicBarrier, all participating threads typically call await(), whereas in CountDownLatch, the waiting thread(s) and the counting-down thread(s) are often different., Include that CyclicBarrier may execute an optional barrier action once all parties arrive.

Detailed Feedback:

The candidate gave a solid and mostly accurate comparison of CountDownLatch vs CyclicBarrier in Java. They correctly identified the most important distinction: CountDownLatch is one-time use and cannot be reset after the count reaches zero, while CyclicBarrier is reusable and resets after all participating threads reach the barrier. They also correctly explained the coordination pattern difference in practical terms: with CountDownLatch, one or more threads can wait for other threads to finish work, whereas with CyclicBarrier, all participating threads wait for each other at a common synchronization point before continuing. The examples they gave were relevant and technically appropriate. What keeps this from being a very high score is that the explanation, while correct, was somewhat narrow and omitted a few useful comparison details that are part of a fuller answer to 'how they differ' in Java. In particular, they did not explicitly mention that CountDownLatch uses a decrementing count typically driven by worker completion, while CyclicBarrier is based on a fixed number of parties that must all call await(). They also did not mention barrier break behavior/interruption handling, though that is not strictly required unless deeper detail is expected. Overall, this is a good answer that directly addresses the question with correct core distinctions.

Q18: A query returns duplicate rows when joining multiple tables. What is the MOST efficient solution?

Category: SQL & Data Handling | Question Category: Core | Difficulty: medium | Type: OBJECTIVE

Candidate's Verbal Answer:

Review join condition

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'Review join condition' is correct.

Q19: Explain API Gateway pattern and its benefits.

Category: Microservices | Question Category: Core | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

> Mainly API gateways patterns. It is a single entry points for all clients request instance of clients calling 10 microservices directly. They call one gateway and gateway routes to internal services. It coming to benefits single entry point and routing and aggregations and third is cross cutting, convoy in one place, securities and proto-transactions can be handling rest gRPC, web, sockets, conversions.

Score: 74/100

Good: Correctly identified the core architectural idea of the API Gateway pattern, Mentioned multiple valid benefits rather than only defining the pattern, Included routing and aggregation, which are important gateway responsibilities, Recognized centralization of security and protocol handling

Improve: State the definition more clearly: an API Gateway is a server that sits between clients and backend services and acts as the single entry point for requests., Make the benefits more explicit: it reduces client complexity, hides internal microservice structure, and centralizes concerns like authentication/authorization, rate limiting, logging, and protocol translation., Use correct and clear terminology for unclear phrases such as "convoy" and "proto-transactions"; if the intent was cross-cutting concerns and protocol translation, say that directly.

Detailed Feedback:

The candidate gave a substantively correct explanation of the API Gateway pattern and covered the main benefits that the question asked for. They correctly described it as a single entry point in front of multiple microservices, where clients call the gateway instead of calling many services directly. They also correctly mentioned routing and aggregation, which are core parts of the pattern. In addition, they identified cross-cutting concerns being centralized in one place, including security, and they appeared to mention protocol translation/conversion across REST, gRPC, and WebSockets, which is also a valid benefit. There are some weaknesses. The explanation is fragmented and not very clearly structured, and a few phrases are likely transcription errors or unclear speech (for example, "convoy" and "proto-transactions"). Interpreting generously, they likely meant common cross-cutting concerns and protocol translation, but those parts were not articulated cleanly. Also, while they listed benefits, they did not clearly explain why those benefits matter to clients and services—for example, reducing client complexity, hiding internal service topology, or enabling centralized authentication/rate limiting. Still, since the question only asked to explain the pattern and its benefits, not to provide deep architecture tradeoff analysis, the answer is reasonably complete. Overall, this is a good answer: technically mostly correct, relevant to microservices/API architecture, and it addresses both the pattern and benefits. It falls short of very good because of limited clarity and incomplete articulation of some important benefits.

Q20: What are invisible indexes and how can they be used safely to test index impact before making them visible?

Category: Performance & Security | Question Category: Core | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

> invisible index equal to index maintained but ignored by optimizer visible equal to normal index safe testing instantly of dropping any index make it invisible first if if performance drops make it visible again instantly if no impact then drop it safely it avoids the costly rebuild if you drop and realize you still need it

Score: 82/100

Good: Core definition was correct, Answer stayed relevant to Oracle behavior, Provided a practical and accurate safe-testing workflow, Demonstrated understanding that invisible indexes are still maintained and can be restored instantly

Improve: Include the other key safe-testing pattern: create or keep the index invisible, then enable optimizer_use_invisible_indexes for controlled testing before making it visible to all sessions., State more explicitly that the index remains physically present and maintained; only optimizer visibility changes by default.

Detailed Feedback:

The candidate gave a solid and technically accurate answer for the scope of the question. They correctly defined an invisible index as an index that is still maintained but ignored by the optimizer, which is the core concept in Oracle. They also correctly explained the main safe-use pattern: instead of dropping an index immediately, make it invisible first, observe whether performance regresses, and if needed make it visible again quickly. They also correctly noted the

practical benefit that this avoids having to rebuild the index if you discover it was still needed. What keeps this from a higher score is completeness. The question asked both what invisible indexes are and how they can be used safely to test index impact before making them visible. The candidate covered the safer workflow mainly in the context of testing before dropping an index, which is a valid and common use case, but they did not explicitly mention the complementary Oracle behavior that invisible indexes can also be tested by enabling optimizer use of invisible indexes at the session/system level before making them visible to everyone. That is an important part of safely testing index impact before exposing the index broadly. So the answer is accurate, but somewhat narrower than the full question suggests. Overall: good understanding, accurate terminology, and a practical Oracle-relevant use case, but missing one important safe-testing mechanism.

Q21: Implement infinite scroll with performance optimization.

Category: React:Optimization | Question Category: Core | Difficulty: medium | Type: CODE

Candidate's Code Answer:

```
const observer = useRef();
const lastRef = useCallback(node =>{
if (loading) return ;
observer.current ?.disconnect();
observer.current = new IntersectionObserver(e=> {
if (e[0].isIntersecting && hasMore) setPage(p=> p+1);

});
if (node) observer.current.observe(node);
},[loading,hasMore]);
```

Score: 62/100

Good: Uses IntersectionObserver, which is an appropriate performance optimization over scroll listeners, Disconnects the previous observer before creating a new one, Uses functional state update for page increment, Guards against triggering while loading, Checks `hasMore` before incrementing pagination

Improve: Add proper TypeScript types for the ref, observed node, and observer entries, Provide explicit cleanup on unmount to avoid lingering observer instances, Include observer options such as `rootMargin` or `threshold` to preload earlier and reduce visible loading gaps, Prevent duplicate page increments in edge cases where intersections fire repeatedly before loading state updates, Show the surrounding fetch/state flow to make the solution complete, Handle null/undefined state assumptions more explicitly

Detailed Feedback:

This is a partial but generally correct React infinite-scroll snippet. The core mechanism is valid: it uses an IntersectionObserver tied to the last element, disconnects the previous observer, and increments page state when the sentinel becomes visible and more data exists. That is a reasonable performance-oriented approach compared to scroll event listeners. However, it is not a complete implementation of infinite scroll. It assumes surrounding state and data-fetching logic exist, provides no cleanup on unmount beyond per-callback disconnect, has weak typing, and does not address several practical performance concerns such as observer options, duplicate triggers during rapid intersections, request throttling/debouncing strategy, or stable sentinel handling. In React/TypeScript terms, `useRef()` is untyped, `node` is untyped, and the observer callback parameter is untyped. The solution works as a fragment, but not as a robust full answer to the stated problem. ■■ Note: Compilation failed due to environment/infrastructure issues (e.g. missing JUnit/Spring jars in the runner). No score penalty applied. Compiler output: /usr/local/lib/node_modules/ts-node/node_modules/arg/index.js:90 throw err; ^ Error: Unknown or unexpected option: --allowJs at arg (/usr/local/lib/node_modules/ts-node/node_modules/arg/index.js:88:19) at parseArgv (/usr/local/lib/node_modules/ts-node/dist/bin.js:69:12) at main (/usr/local/lib/node_modules/ts-node/dist/bin.js:25:18) at Object. (/usr/local/lib/node_modules/ts-node/dist/bin.js:579:5) at Module._compile (node:internal/modules/cjs/loader:1521:14) at Module._extensions..js (node:internal/modules/cjs/loader:1623:10) at Module.load (node:internal/modules/cjs/loader:1266:32) at Module._load (node:internal/modules/cjs/loader:1091:12) at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:164:12) at node:internal/main/run_main_module:28:49 { code: 'ARG_UNKNOWN_OPTION' } Node.js v20.20.2

Q22: Write a query to retrieve all employees from the EMPLOYEES table who work in department 50 and have a salary greater than 5000. Sort the results by salary in descending order.

Category: Advance SQL | Question Category: Foundational | Difficulty: hard | Type: CODE

Candidate's Code Answer:

```
SELECT * FROM EMPLOYEES
WHERE DEPARTMENT_ID = 50
AND SALARY > 5000
ORDER BY SALARY DESC;
```

Score: 88/100

Good: Uses the correct table and required filters, Applies the correct comparison operator for salary greater than 5000, Sorts results in descending salary order as requested, Query is concise and easy to read

Improve: Avoid SELECT * and project only the needed columns, Consider explicit column selection for better maintainability and performance

Detailed Feedback:

This correctly solves the stated problem. It filters EMPLOYEES to department 50, applies the salary > 5000 condition, and sorts by salary descending. Semantically, the query matches the requirements exactly. The main issue is quality rather than correctness: using SELECT * is weaker practice than selecting only required columns, especially in production SQL, because it can return unnecessary data and makes the query more brittle to schema changes. For this specific prompt, though, the result set is functionally correct and complete. ■■ Note: Compilation failed due to environment/infrastructure issues (e.g. missing JUnit/Spring jars in the runner). No score penalty applied. Compiler output: psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: No such file or directory Is the server running locally and accepting connections on that socket?

Q23: How would you render a large number of markers (10,000+) on a map without causing performance issues?

Category: React:Maps | Question Category: Core | Difficulty: hard | Type: VERBAL

Candidate's Verbal Answer:

Coming to rendering 10k DOM marks will kills performance I will use 3 technologies here one is clustering clustering using supercluster or market clustering group near by markers into cluster marker reduce DOM needs destination and coming to viewpoint rendering it will be it will be used only for the rendering marking inside current map bounds on bounds changed filters marks like 10k may be 100 visible and coming to third canvas it will be also used it will be also known as WebGL layer instead of 10k divs using dck.gi or Google Maps advanced marker elements with canvas overlay dot 1 canvas equal to 10k points in one point

Score: 82/100

Good: Answered the actual question directly, Provided multiple valid scaling strategies rather than only one, Correctly identified clustering as a primary optimization, Correctly identified viewport culling as an important optimization, Correctly recognized that Canvas/WebGL scales better than thousands of DOM nodes, Included concrete tooling examples like Supercluster and deck.gl

Improve: Be more precise when describing rendering technologies: Canvas/WebGL layers are the scalable alternative, whereas AdvancedMarkerElement is not equivalent to a true canvas/WebGL solution for 10k+ markers., State the strategy more clearly as: cluster at low zoom, render only markers within current bounds, and use Canvas/WebGL for very large datasets instead of individual React/DOM markers., Since the question is in React context, explicitly mention avoiding unnecessary React re-renders and keeping marker rendering out of the normal DOM/component tree when possible.

Detailed Feedback:

The candidate gave a solid practical answer to the actual question: how to render 10,000+ markers on a map without performance issues in a React context. The core ideas were correct and relevant. They identified three strong strategies: marker clustering (for reducing the number of rendered marker elements), viewport/bounds-based rendering (only render markers currently visible on the map), and switching from many DOM-based markers to a canvas/WebGL-based rendering approach for very large datasets. Those are exactly the main performance techniques expected here. Technically, the answer shows real understanding of why performance suffers: rendering 10k DOM markers is expensive. Mentioning Supercluster was a strong concrete detail. The explanation of filtering by current map bounds was also correct and useful. The canvas/WebGL point was directionally correct as well: for very large numbers of points, drawing via Canvas or WebGL is much more scalable than rendering thousands of React/DOM nodes. There are some weaknesses that prevent this from being a very high score. The answer was somewhat fragmented and

imprecise in places. "Google Maps advanced marker elements with canvas overlay" is muddled: AdvancedMarkerElement itself is not the same thing as a canvas/WebGL rendering strategy, and for 10k+ points it is generally still not the best answer compared with true canvas/WebGL layers. Also, "1 canvas equal to 10k points in one point" was unclear, though the intended meaning was understandable. In React specifically, they did not explicitly mention reducing React re-renders, memoization, or keeping map rendering outside normal React DOM reconciliation, which would have strengthened the React-specific framing. However, the question asked how to render large numbers of markers efficiently, and they did answer that directly. Overall, this is a good answer: technically relevant, mostly accurate, and complete enough for the scope of the question, but not polished or fully precise.

Q24: Explain the "String Pool". Why are Strings immutable in Java?

Category: Memory Management | Question Category: Foundational | Difficulty: easy | Type: VERBAL

Candidate's Verbal Answer:

> StringFool is a special memory area inside hoop where java stores string literally to save memory. If you create string1 equal to hello, it goes to pool. If another string is to equal to hello, it receives same object, no new object. Why strings are immutable means it will be security. That means strings are used for dburls file path if mutable can be changed. StringFool needs it. If mutable once change would affect you, all reference sharing it. I am coming to third thread safe and hashmap keys. Hashcode is catchable and to save as hashmap key. Yeah, that's it.

Score: 74/100

Good: Answered both parts of the question directly, Correctly identified memory saving through shared literals in the pool, Correctly connected immutability to pooled object sharing, Included multiple valid reasons for String immutability beyond just one memorized point, Demonstrated practical Java knowledge with security and HashMap key usage

Improve: Define immutability explicitly: a String object cannot be modified after creation; operations create a new String instead., State more precisely that the String Pool is the JVM's intern pool for string literals and interned strings., Clarify the HashMap point: Strings are good keys because immutable values keep hashCode and equality behavior consistent after insertion., Use the correct term 'String Pool' instead of the transcribed 'StringFool'; this appears to be a transcription issue, not necessarily a knowledge issue.

Detailed Feedback:

The candidate shows solid practical understanding of both parts of the question. Interpreting the transcription, they meant 'String Pool' as a special memory area in the heap where Java stores string literals so identical literals can share the same object and save memory. That is essentially correct. They also correctly explained that if two string literals have the same value, Java can reuse the same pooled String object instead of creating a new one. For immutability, they gave several valid reasons: security (for things like file paths / DB URLs), safe sharing of pooled strings, thread-safety benefits, and stable behavior as HashMap keys due to cached hash codes. These are all legitimate reasons Java Strings are immutable. The point about pooled strings especially is important: if Strings were mutable, changing one shared reference would affect others pointing to the same pooled object. However, the answer is not fully complete or fully precise. The candidate did not clearly distinguish that the String Pool is specifically the intern pool for string literals and interned strings, rather than all Strings in general. Saying 'inside heap' is acceptable in modern JVM terms, but the explanation could have been more precise. The HashMap point was somewhat garbled ('hashcode is catchable') but the intended meaning was clear: immutable Strings can safely cache hashCode and remain reliable as map keys. Also, the immutability explanation was more of a list of reasons than a structured explanation. Overall, this is a good answer: technically mostly correct, addresses both parts of the question, and demonstrates real understanding. It falls short of very good because of imprecision and lack of a cleaner explanation of what the pool actually is and how interning relates to it.

Q25: What is optimistic updates and how do you implement them?

Category: React:TanStack Query | Question Category: Core | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

> Optimized update means we update UI immediately assuming API will success before getting services response it makes app feel instance like Instagram like and coming to in coming to uses means uses mutual I use only for mutated there I can cancel out outgoing queries snap snapshots previously data and set query data with new optimistic values if API fails in on error I roll back to snapshot if success in one settlement I invalidate queries to sync with real service data

Score: 82/100

Good: Correct definition of optimistic updates, Accurately described the rollback mechanism, Included concrete implementation details rather than only a vague definition, Used a valid React ecosystem approach via React Query mutation handlers, Understood the need to resync with server state after the mutation completes

Improve: Structure the answer more clearly: first define optimistic updates, then explain implementation step by step, Mention that in plain React, the same idea can be implemented by updating local state immediately, calling the API, and reverting on failure, State more explicitly that optimistic updates are mainly used for mutations such as add/edit/delete actions

Detailed Feedback:

The candidate demonstrates solid understanding of optimistic updates in React, especially in the common React Query/TanStack Query implementation pattern. They correctly define the concept as updating the UI immediately before the API response returns, assuming success, and they correctly explain the rollback flow if the request fails. They also mention key implementation steps: canceling outgoing queries, taking a snapshot of previous data, setting optimistic cache data, rolling back in an error handler, and invalidating queries afterward to resync with server state. That is technically strong and directly answers both parts of the question: what it is and how to implement it. The main limitation is that the answer is somewhat tool-specific and fragmented. They appear to describe optimistic updates specifically using React Query mutation lifecycle callbacks rather than explaining the broader React-level implementation approach as well (e.g. local state update first, then revert on failure). Since the question is in React context, this is still valid, but the explanation could have been clearer and more structured. There are also transcription issues and some unclear phrasing, but the technical intent is understandable. Overall, this is a good answer: accurate, relevant, and sufficiently complete for the question asked, though not especially polished or broadly framed.

Q26: What is "Escape Analysis"? How can it lead to stack allocation of objects?

Category: JVM | Question Category: Foundational | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

Escape analysis is a JVM optimization. JVM actually checks if an object created inside a method escapes the method like returning it or passing it to another thread. If it does not escape, JVM knows the object is only used inside the method. It can optimize by allocating it on stack instead of heap. Stack allocation is super cheap and no GC needed. Object dies when method ends. It can also do scalable replacement, break object into primitive fields. That's why local short lived object does not always

Score: 88/100

Good: Answered both parts of the question directly, Technically accurate explanation of what escape analysis checks, Gave concrete examples of what it means for an object to escape, Correctly connected escape analysis to allocation optimization and GC reduction, Mentioned scalar replacement, showing deeper JVM optimization knowledge

Improve: Clarify that escape analysis does not guarantee stack allocation; it allows the JIT to optimize non-escaping allocations, often via scalar replacement or allocation elimination, Use the correct term "scalar replacement" explicitly

Detailed Feedback:

This is a very good answer. The candidate correctly identifies escape analysis as a JVM optimization that determines whether an object allocated within a method can be observed outside that scope, and they give valid examples of escaping behavior such as returning the object or sharing it with another thread. They also correctly explain the key consequence relevant to the question: if the object does not escape, the JVM may optimize it by avoiding heap allocation and effectively allocating it on the stack, which reduces GC pressure and allows the object to die with the method/frame. They also mention scalar replacement, which is an important related optimization; the transcribed phrase "scalable replacement" is clearly a speech-to-text error for "scalar replacement," so this should be treated as correct. Relative to the exact question asked, the answer is largely complete and technically accurate. The only notable gap is that the candidate states stack allocation somewhat definitively ("it can optimize by allocating it on stack") without clarifying that in HotSpot/JIT this is an optimization opportunity rather than a guaranteed outcome, and in practice escape analysis often enables scalar replacement or allocation elimination rather than literal stack allocation in every case. Still, for an interview-level verbal explanation, this is a strong answer and clearly demonstrates understanding. The final sentence is cut off, so the answer feels slightly incomplete, but the core substance needed to answer the question was already provided.

Q27: When is REST better than Message-based

Category: API Design | Question Category: Foundational | Difficulty: easy | Type: VERBAL

Candidate's Verbal Answer:

> Rest is better when you need scenario synchronize request response like CRUD operations, users facing APIs, simple client service where immediately response needed. Message based like Kafka, Rabbit, MQ is better for async heavily background jobs, event driven. So use restful for simple, immediately synchronize operations, use messaging when you need decomposing,

Score: 78/100

Good: Answered the comparison directly rather than giving a generic definition, Technically correct distinction between synchronous REST and asynchronous messaging, Used relevant real-world examples of messaging technologies, Demonstrated practical architectural understanding

Improve: Make the REST side of the comparison more explicit: REST is better when the caller needs an immediate response and the operation is straightforward request-response., Clarify the contrast with messaging by explicitly stating that messaging is preferred when decoupling, queueing, or eventual consistency is needed., Replace vague wording like 'when you need decomposing' with precise architectural terms such as decoupling or asynchronous event processing.

Detailed Feedback:

The candidate gave a solid high-level comparison and answered the question directly. They correctly identified that REST is preferable for synchronous request-response interactions, especially CRUD-style operations and user-facing APIs where an immediate response is needed. They also correctly contrasted this with message-based systems such as Kafka/RabbitMQ for asynchronous, background, and event-driven processing. That is the core of what the question asked. However, the answer is somewhat brief and incomplete for a comparison question phrased as 'When is REST better than message-based'. A stronger answer would have more explicitly stated that REST is better when the caller needs an immediate result, when the interaction is simple and tightly scoped, when the operation maps naturally to resource-oriented APIs, and when the workflow does not require decoupling, buffering, retries, or eventual consistency. The phrase 'when you need decomposing' is vague and not clearly explained. Also, while 'synchronize' appears to be a transcription artifact for 'synchronous', the intended meaning is clear and technically correct. Overall, this shows good practical understanding, but not a comprehensive comparison.

Q28: How do you implement the "Singleton Pattern" correctly to make it thread-safe and reflection-proof?

Category: Design Patterns | Question Category: Foundational | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

> single single turn thread safely under reflex reflection proof is to use enum enum single term is by default thread safe and JVM preventive reflection from creating another instance if you have to use class use double checked look with toilet instance make constructor provide and throw exception if instance already exists to block reflection also implement read resolve to block serialization creating new instant

Score: 88/100

Good: Identified enum as the best Java solution, Covered both thread safety and reflection concerns directly, Included serialization protection via readResolve for class-based singleton, Demonstrated practical Java-specific knowledge rather than generic design-pattern theory

Improve: Clarify that enum is the strongest correct answer in Java because class-based singletons are difficult to make fully reflection-proof., When describing double-checked locking, explicitly mention the synchronized block in addition to volatile., Be precise that readResolve solves serialization duplication, which is separate from reflection-proofing.

Detailed Feedback:

The candidate gave a strong and technically accurate answer for Java. They correctly identified the best practical solution: using an enum singleton, which is inherently thread-safe and resistant to reflection-based instantiation in normal Java usage. They also covered the alternative class-based implementation with double-checked locking on a volatile instance, a private constructor, and a constructor guard to reduce reflection risk. They additionally mentioned readResolve to prevent serialization from creating a second instance, which is an important correctness detail for non-enum singletons. This answer addresses the actual question well: thread safety and reflection-proofing. The enum recommendation is the strongest part and is the most correct concise answer in Java. The class-based alternative is also mostly correct. One nuance: saying a constructor check fully makes a class-based singleton reflection-proof is slightly too strong. Reflection can still invoke the constructor before normal initialization in some cases, so a class-based singleton is harder to make truly reflection-proof than an enum. In Java, enum is the cleanest answer for both thread safety and reflection resistance. Also, if using double-checked locking, the candidate should explicitly say

volatile instance and synchronized block, though "double checked lock with volatile instance" strongly implies that. Overall, this is a very good answer: accurate, relevant, and complete for the scope of the question, with only minor nuance missing around the limits of reflection defense in class-based singletons.

This report was generated by the Evalo.
© 2026 IRIS Software. All rights reserved.